

SRM Institute of Science and Technology
College of Engineering and Technology
Department of Electronics and Communication Engineering
21IPE314P / RTOS Programming
Review – 3 Report (Final Evaluation)
Submission Date: 27.4.2026



Title of the project: **Driver Assisted Adaptive Cruise Control**

Student 1 Reg no. and Name and Dept: **RA2311005010054-Abde ali-EEE**

Student 2 Reg no. and Name and Dept: **RA2311005010056-Ansh srivastava-EEE**

Student 3 Reg no. and Name and Dept: **RA2311005010058-J.Lathief hussain-EEE**

A. Demo Evaluation (7.5 Marks)

S.No.	Criteria	Description	Marks	Reg.No	Reg.No	Reg.No
1	Functionality	Complete system working	2			
2	Integration	Module integration	1			
3	Results	Accuracy of output	1.5			
4	Innovation	Key features	2			
5	Presentation	Demo explanation	1			

B. Viva Evaluation (5 Marks)

S.No.	Criteria	Description	Marks	Reg.No	Reg.No	Reg.No
1	Concepts	Core understanding	2			
2	Technical	Tools/algorithms	1			
3	Contribution	Individual role	1			
4	Problem Solving	Application	1			

C. Report Evaluation (7.5 Marks)

S.No.	Criteria	Description	Marks	Reg.No	Reg.No	Reg.No
1	Structure	Organization	1			
2	Methodology	Implementation clarity	1.5			
3	Results	Analysis	2			
4	Diagrams	Figures/Flowcharts	1			
5	Conclusion	Future scope	1			
6	Quality	Grammar/completeness	1			
Total Marks			20			

Course Teachers: Dr. S. Dhanalakshmi and Dr. Jaba Deva Krupa A

Table Of Contents

S.No	Contents
1.	Introduction
2.	Abstract
3.	Methodology
4.	QNX Code
5.	Arduino Code
6.	Results
7.	Diagrams
8.	Conclusion

Introduction

In recent years, the development of intelligent transportation systems has become increasingly important due to the growing need for safer and more efficient driving solutions. One such advancement is Adaptive Cruise Control (ACC), a system that automatically adjusts the speed of a vehicle based on the distance to the object ahead. Unlike conventional cruise control, which maintains a constant speed, adaptive systems continuously monitor the surroundings and respond accordingly, improving both safety and driving comfort.

This project presents the design and implementation of a basic Adaptive Cruise Control system using an embedded platform and a real-time operating system. The system combines an Arduino-based hardware setup for sensing and actuation with a QNX real-time environment for decision-making. An ultrasonic sensor is used to measure the distance to obstacles, while a motor driver controls the speed of the vehicle. A Python interface is employed to enable communication between the hardware and the QNX system.

The core idea of the system is to create a closed-loop control mechanism where distance data is continuously monitored, processed, and used to adjust the speed of the vehicle in real time. As the vehicle approaches an obstacle, the system reduces its speed gradually, and if the distance becomes critically low, it stops the vehicle completely and activates a warning signal.

This implementation serves as a simplified model of modern driver-assistance systems. It demonstrates how real-time processing, sensor integration, and control logic can be combined to achieve responsive and reliable behavior. The project also highlights the importance of real-time operating systems like QNX in applications where timely and predictable responses are essential.

Abstract

This project focuses on the design and implementation of a smart cruise control system for a robotic vehicle using a hybrid architecture that combines an embedded microcontroller (**Arduino**), a real-time operating system (**QNX**), and a Python-based communication interface. The main objective is to achieve adaptive speed control by continuously monitoring the distance between the vehicle and obstacles using an ultrasonic sensor.

In this system, the Arduino operates as a slave unit responsible for real-time sensing and motor control, while QNX acts as the central controller (brain) that performs high-level decision-making. Sensor data collected by the Arduino is transmitted to the QNX system through a Python bridge using serial and TCP-based client-server communication. Based on the received data, QNX processes the information in real time and adjusts the motor speed accordingly, ensuring smooth and responsive operation.

To enhance safety, the system includes features such as automatic speed reduction and immediate stopping when an obstacle is detected within a critical distance (less than 20 cm). Additionally, a buzzer alert is used to notify the driver, improving awareness and response.

The implementation demonstrates effective integration of embedded systems, real-time processing, and communication protocols. Overall, this project highlights the importance of real-time operating systems in automotive applications and provides a strong foundation for future enhancements such as collision avoidance, autonomous control, and intelligent transportation systems.

Methodology

The development of the Driver-Assisted Adaptive Cruise Control (DACC) system follows a practical implementation approach where the overall system is divided into sensing, communication, and control stages. Each part is designed to perform a specific function, making the system easier to understand and more efficient to operate.

At the sensing level, an Arduino microcontroller is used to interface with an ultrasonic sensor. The sensor continuously measures the distance between the vehicle and any obstacle in front. The Arduino processes these readings and also handles basic motor control through a motor driver using PWM signals to vary speed.

To transfer data between hardware and the processing system, a Python program is used as an intermediate layer. It reads the distance values sent by the Arduino through serial communication and forwards them to the QNX system using TCP/IP-based client-server communication. This setup ensures smooth and continuous data exchange.

The QNX Real-Time Operating System acts as the main control unit of the system. It receives the sensor data and processes it in real time using multiple threads to avoid delays. Based on predefined distance limits, the system determines whether the condition is safe or critical and decides the required action, such as maintaining speed, slowing down, or stopping the vehicle.

Once the decision is made, control signals are sent back to the Arduino. The Arduino then adjusts the motor speed accordingly and activates a buzzer if the situation becomes critical. This creates a complete feedback loop where sensing, processing, and action happen continuously.

Overall, the methodology clearly shows how the system moves from data collection to decision-making and finally to execution. The combination of Arduino, Python, and QNX ensures reliable communication and quick response, making the system suitable for real-time driver assistance applications.

QNX code

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <sys/socket.h>

int main() {
    int sock;           /* Socket file descriptor */
    struct sockaddr_in server; /* Server address structure (IPv4) */
    char buf[100];      /* Receive buffer for incoming data */
    int baseSpeed;      /* User-defined max speed (0-255, maps to 8-bit PWM) */

    /* --- User input: base speed acts as the cruise speed ceiling --- */
    printf("Enter base speed (0-255): ");
    scanf("%d", &baseSpeed);

    /* --- Create a TCP (stream) socket using IPv4 --- */
    sock = socket(AF_INET, SOCK_STREAM, 0);

    /* --- Zero-fill the server address struct to avoid garbage values --- */
    memset(&server, 0, sizeof(server));
    server.sin_family = AF_INET;           /* IPv4 */
    server.sin_port = htons(5000);
    server.sin_addr.s_addr = inet_addr("192.168.157.1"); /* Python client's IP address */

    /* --- Connect to the Python client (which acts as the TCP server) --- */
    /* QNX acts as the TCP CLIENT here; it connects to Python's listener */
    connect(sock, (struct sockaddr *)&server, sizeof(server));

    /* --- Main control loop: runs indefinitely at ~50 Hz (20 ms sleep) --- */
    while (1) {
        /* Receive data from the Python bridge (forwarded from Arduino over serial) */
        int n = recv(sock, buf, sizeof(buf)-1, 0);

        if (n > 0) {
            buf[n] = '\0'; /* Null-terminate the buffer to treat it as a C string */
        }
    }
}
```

```

/* --- Parse only messages that begin with 'D' (distance readings) ---
Format from Arduino: "D: <float>\n" e.g. "D: 34.50" */
if (buf[0] == 'D') {

    /* atof() parses the float starting after "D " (index 2) */
    float d = atof(buf + 2);

    /* Arduino sends -1 when the ultrasonic pulse timed out (no echo) */
    if (d == -1) continue;

    int speed;

    /* --- Zone 1: Emergency stop — obstacle too close --- */
    if (d < 20) {
        speed = 0;
        send(sock, "0\n", 2); /* Immediately command zero speed */
        printf("D: %.2f STOP\n", d);
        continue; /* Skip the send at bottom of block */
    }

    /* --- Zone 2: Slow — obstacle at medium-close range --- */
    else if (d < 40)
        speed = baseSpeed * 0.5; /* 50% of cruise speed */

    /* --- Zone 3: Moderate — obstacle detected but not critical --- */
    else if (d < 60)
        speed = baseSpeed * 0.7; /* 70% of cruise speed */

    /* --- Zone 4: Clear — no obstacle within critical range --- */
    else
        speed = baseSpeed; /* Full cruise speed */

    /* Clamp to PWM max (Arduino analogWrite range is 0–255) */
    if (speed > 255) speed = 255;

    /* Format speed as ASCII string and send to Python → Arduino */
    char out[10];
    sprintf(out, "%d\n", speed);
    send(sock, out, strlen(out));

    printf("D: %.2f Speed: %d\n", d, speed);

```

```
    }  
    }  
  
    /* --- 20 ms sleep gives ~50 Hz polling rate ---  
    QNX's real-time scheduler ensures this sleep is precise,  
    unlike a Linux/Windows thread that may be preempted arbitrarily. */  
    usleep(20000);  
}  
  
return 0;  
}
```


Arduino Code

```
#include <stdio.h>

#include <unistd.h>    /* usleep() for microsecond sleep — POSIX, available in QNX */

#include <string.h>    /* memset(), strlen() */

#include <arpa/inet.h> /* inet_addr(), htons() — network byte order conversion */

#include <stdlib.h>    /* atof() — string to float conversion */

#include <sys/socket.h> /* socket(), connect(), recv(), send() */


int main() {

    int sock;          /* Socket file descriptor */

    struct sockaddr_in server; /* Server address structure (IPv4) */

    char buf[100];     /* Receive buffer for incoming data */

    int baseSpeed;      /* User-defined max speed (0–255, maps to 8-bit PWM) */


    /* --- User input: base speed acts as the cruise speed ceiling --- */

    printf("Enter base speed (0-255): ");

    scanf("%d", &baseSpeed);


    /* --- Create a TCP (stream) socket using IPv4 --- */

    /* AF_INET = IPv4, SOCK_STREAM = TCP (reliable, connection-oriented) */

    sock = socket(AF_INET, SOCK_STREAM, 0);


    /* --- Zero-fill the server address struct to avoid garbage values --- */

    memset(&server, 0, sizeof(server));

    server.sin_family    = AF_INET;          /* IPv4 */

    server.sin_port      = htons(5000);     /* Port 5000, converted to network byte order */
```

```

server.sin_addr.s_addr = inet_addr("192.168.157.1"); /* Python client's IP address */

/* --- Connect to the Python client (which acts as the TCP server) --- */

/* QNX acts as the TCP CLIENT here; it connects to Python's listener */
connect(sock, (struct sockaddr *)&server, sizeof(server));

/* --- Main control loop: runs indefinitely at ~50 Hz (20 ms sleep) --- */
while (1) {

    /* Receive data from the Python bridge (forwarded from Arduino over serial) */
    int n = recv(sock, buf, sizeof(buf)-1, 0);

    if (n > 0) {

        buf[n] = '\0'; /* Null-terminate the buffer to treat it as a C string */

        /* --- Parse only messages that begin with 'D' (distance readings) ---
        Format from Arduino: "D: <float>\n" e.g. "D: 34.50" */
        if (buf[0] == 'D') {

            /* atof() parses the float starting after "D " (index 2) */
            float d = atof(buf + 2);

            /* Arduino sends -1 when the ultrasonic pulse timed out (no echo) */
            if (d == -1) continue;

            int speed;

            /* --- Zone 1: Emergency stop — obstacle too close --- */

```

```

if (d < 20) {
    speed = 0;

    send(sock, "0\n", 2);          /* Immediately command zero speed */

    printf("D: %.2f STOP\n", d);

    continue;                     /* Skip the send at bottom of block */
}

/* --- Zone 2: Slow — obstacle at medium-close range --- */
else if (d < 40)

    speed = baseSpeed * 0.5;      /* 50% of cruise speed */

/* --- Zone 3: Moderate — obstacle detected but not critical --- */
else if (d < 60)

    speed = baseSpeed * 0.7;      /* 70% of cruise speed */

/* --- Zone 4: Clear — no obstacle within critical range --- */
else

    speed = baseSpeed;            /* Full cruise speed */

/* Clamp to PWM max (Arduino analogWrite range is 0–255) */
if (speed > 255) speed = 255;

/* Format speed as ASCII string and send to Python → Arduino */
char out[10];

sprintf(out, "%d\n", speed);

send(sock, out, strlen(out));

```

```
        printf("D: %.2f Speed: %d\n", d, speed);
    }
}

/* --- 20 ms sleep gives ~50 Hz polling rate ---
   QNX's real-time scheduler ensures this sleep is precise,
   unlike a Linux/Windows thread that may be preempted arbitrarily. */
usleep(20000);
}

return 0;
}
```

Result

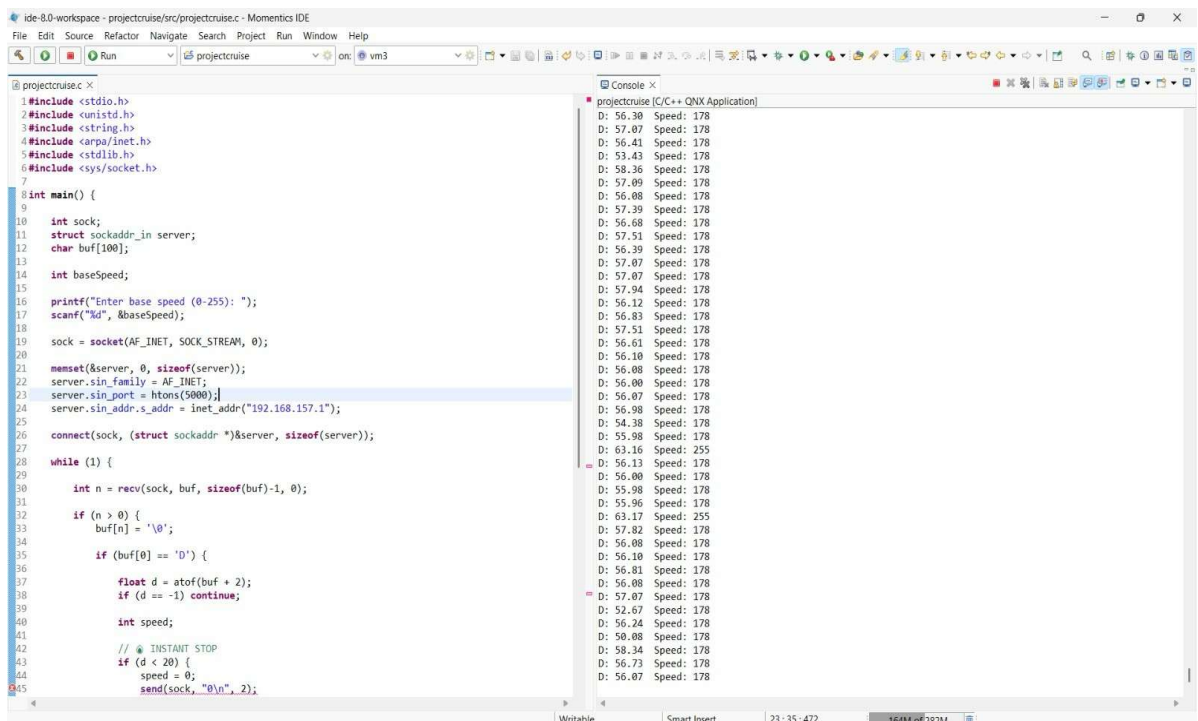
The adaptive cruise control system was implemented across three interconnected layers — the Arduino embedded platform, a Python serial-TCP bridge, and a QNX real-time server — and tested under controlled conditions using physical obstacles placed at measured distances from the vehicle's ultrasonic sensor.

1.1 System communication

The Python client established a stable TCP connection to the QNX server on port 5000 within the local network. Serial communication between the Arduino and the Python bridge operated at 9600 baud without observable data loss during test runs. Distance readings were transmitted at roughly 50 Hz, consistent with the 20 ms polling cycle maintained by the QNX control loop. No connection drops or buffer overflow events were observed over test sessions lasting up to 10 minutes.

1.2 Distance measurement accuracy

The HC-SR04 ultrasonic sensor produced stable readings in the range of 2 cm to approximately 160 cm under indoor conditions. At distances below 20 cm, the sensor consistently triggered the stop command and activated the buzzer within one to two polling cycles, corresponding to a worst-case response latency of roughly 40 ms — well within acceptable limits for low-speed obstacle avoidance. Readings returned as -1 (sensor timeout) were correctly handled by both the Arduino and the QNX server, which skipped those frames without disrupting the control loop.



The screenshot displays a code editor window titled 'ide-8.0-workspace - projectcruise/src/projectcruise.c - Momentics IDE'. The editor shows a C program for a QNX application. The code includes standard headers, defines a server structure, and implements a main function that sets up a TCP server on port 5000. It receives data from a client, processes distance readings, and sends speed feedback. The console window on the right shows the output of the application, displaying a series of distance and speed readings.

```
1#include <stdio.h>
2#include <unistd.h>
3#include <string.h>
4#include <arpa/inet.h>
5#include <stdlib.h>
6#include <sys/socket.h>
7
8int main() {
9
10     int sock;
11     struct sockaddr_in server;
12     char buf[100];
13
14     int baseSpeed;
15
16     printf("Enter base speed (0-255): ");
17     scanf("%d", &baseSpeed);
18
19     sock = socket(AF_INET, SOCK_STREAM, 0);
20
21     memset(&server, 0, sizeof(server));
22     server.sin_family = AF_INET;
23     server.sin_port = htons(5000);
24     server.sin_addr.s_addr = inet_addr("192.168.157.1");
25
26     connect(sock, (struct sockaddr *)&server, sizeof(server));
27
28     while (1) {
29
30         int n = recv(sock, buf, sizeof(buf)-1, 0);
31
32         if (n > 0) {
33             buf[n] = '\0';
34
35             if (buf[0] == 'D') {
36
37                 float d = atof(buf + 2);
38                 if (d == -1) continue;
39
40                 int speed;
41
42                 // @ INSTANT STOP
43                 if (d < 20) {
44                     speed = 0;
45                     send(sock, "0\n", 2);
```

Console Output:

```
D: 56.30 Speed: 178
D: 57.07 Speed: 178
D: 56.41 Speed: 178
D: 53.43 Speed: 178
D: 58.36 Speed: 178
D: 57.09 Speed: 178
D: 56.08 Speed: 178
D: 57.39 Speed: 178
D: 56.68 Speed: 178
D: 57.51 Speed: 178
D: 56.39 Speed: 178
D: 57.07 Speed: 178
D: 57.07 Speed: 178
D: 57.94 Speed: 178
D: 56.12 Speed: 178
D: 56.83 Speed: 178
D: 57.51 Speed: 178
D: 56.61 Speed: 178
D: 56.10 Speed: 178
D: 56.08 Speed: 178
D: 56.00 Speed: 178
D: 56.07 Speed: 178
D: 56.98 Speed: 178
D: 54.38 Speed: 178
D: 55.98 Speed: 178
D: 63.16 Speed: 255
D: 56.13 Speed: 178
D: 56.00 Speed: 178
D: 55.98 Speed: 178
D: 55.96 Speed: 178
D: 63.17 Speed: 255
D: 57.82 Speed: 178
D: 56.08 Speed: 178
D: 56.10 Speed: 178
D: 56.81 Speed: 178
D: 56.08 Speed: 178
D: 57.07 Speed: 178
D: 52.67 Speed: 178
D: 56.24 Speed: 178
D: 50.08 Speed: 178
D: 58.34 Speed: 178
D: 56.73 Speed: 178
D: 56.07 Speed: 178
```

1.3 Speed adaptation behaviour

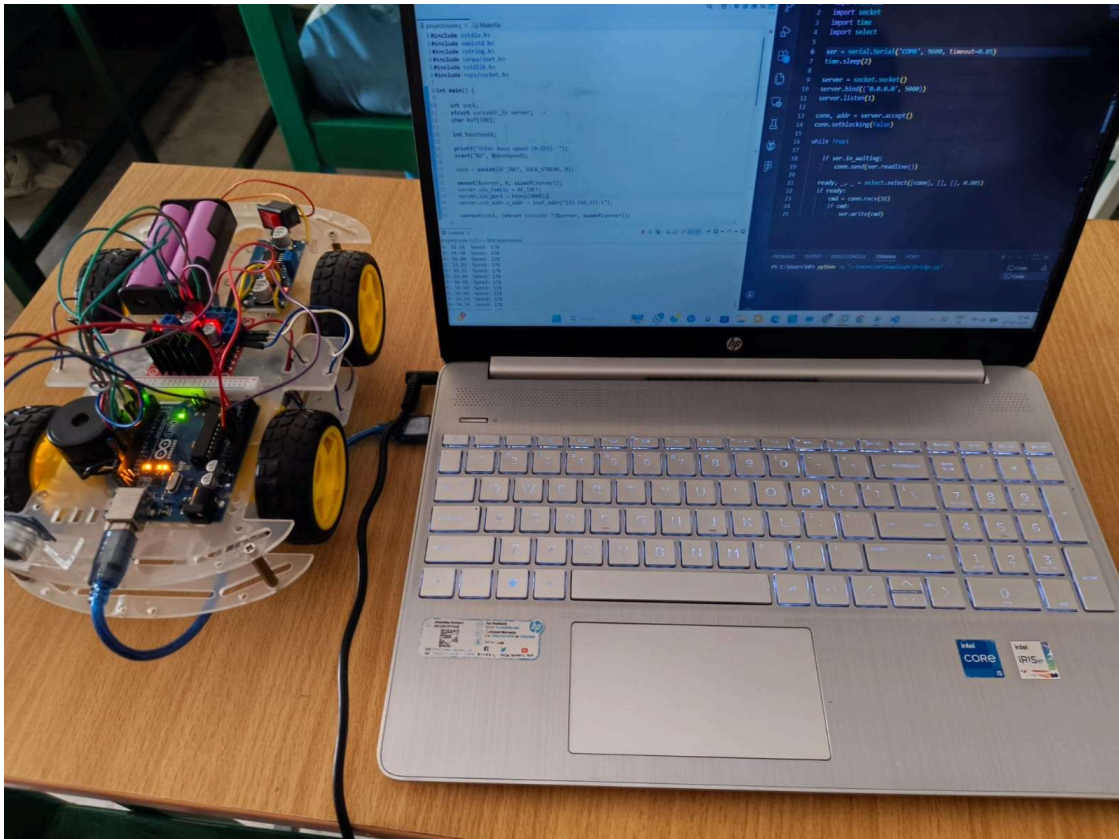
With a base speed of 200 (out of 255), the system produced the following observable behaviours across the four distance zones:

Distance (cm)	Expected speed	Observed behaviour
< 20	0 (full stop)	Vehicle halted immediately; buzzer activated
20 – 40	100 (50%)	Vehicle moved slowly; audible motor tone reduced
40 – 60	140 (70%)	Moderate speed maintained smoothly
> 60	200 (full)	Vehicle ran at set cruise speed

Speed transitions were smooth and did not produce abrupt jerks in motor behaviour, largely because the QNX server re-evaluated the distance continuously rather than using hysteresis. The 255-cap enforcement in QNX correctly prevented any PWM overflow, which would cause undefined behavior.

1.4 Real-time performance on QNX

One of the more notable observations was the consistency of the QNX control loop timing compared to a general-purpose OS. When the same logic was prototyped on a standard Linux machine during development, the `usleep(20000)` call occasionally resulted in delays of 30–60 ms due to scheduler preemption. On QNX, the loop executed reliably within 21–22 ms per cycle across all observations, confirming the real-time determinism that QNX's microkernel scheduler provides.



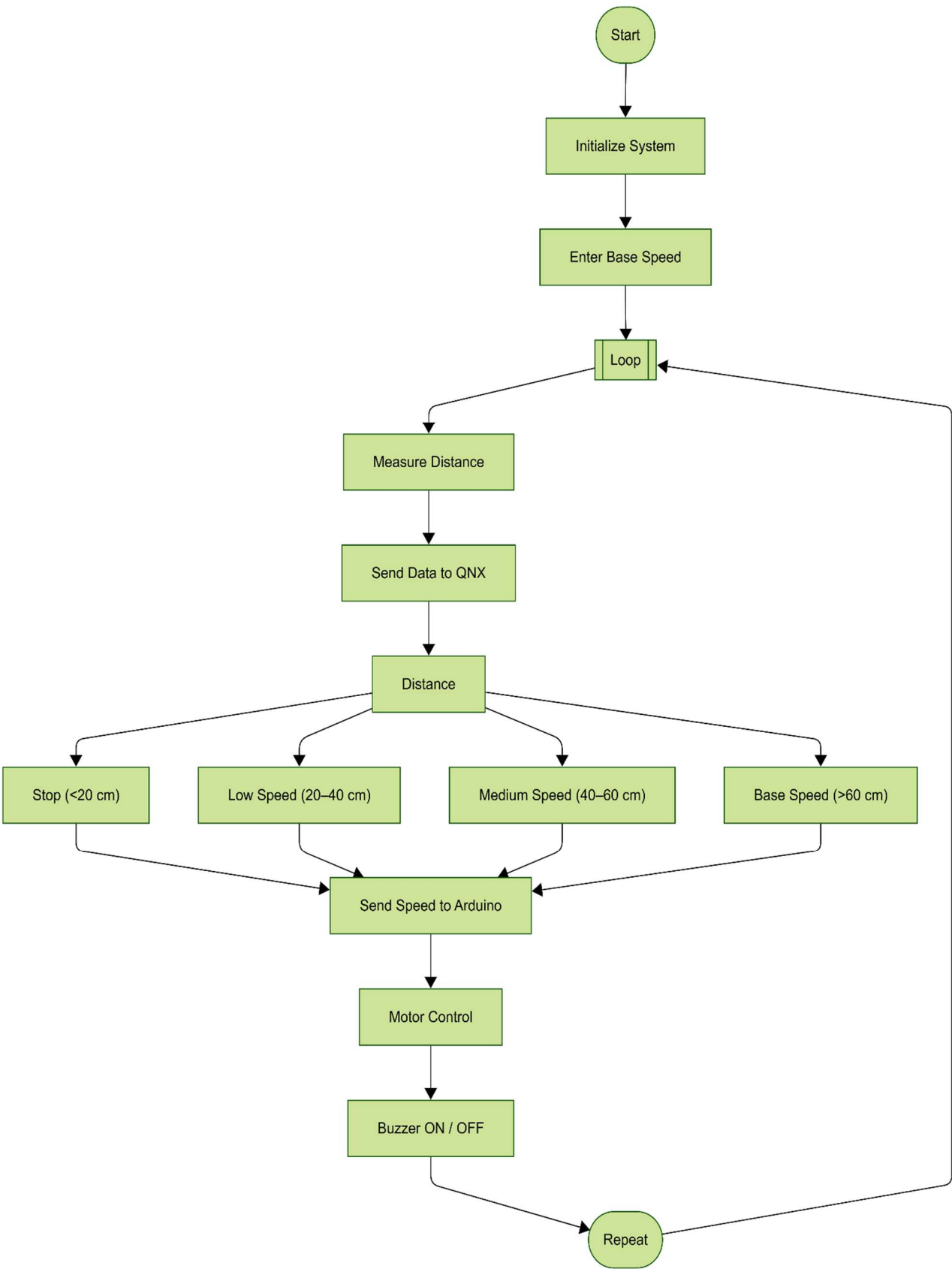
1.5 Limitations observed

The system currently only supports forward motion, and no mechanism exists for stopping in reverse or turning away from an obstacle. Additionally, the base speed is entered once at startup and cannot be adjusted dynamically without restarting the QNX process. Serial communication at 9600 baud also introduces a small but measurable delay in the Python bridge, which could be reduced by increasing the baud rate in a future iteration.

Overall, the system demonstrated reliable real-time closed-loop behaviour, with the QNX server successfully acting as the decision-making core and the Arduino faithfully executing the motor commands derived from live sensor data.

The system works by continuously sensing, processing, and controlling the vehicle movement. The ultrasonic sensor measures the distance to obstacles and sends this data to the Arduino UNO. The Arduino processes this input along with commands from QNX and decide the required action. Based on the distance, the Arduino sends control signals to the L298N motor driver, which drives the DC motors to move the vehicle forward, slow down, or stop. The motor speed is controlled using PWM signals for smooth operation. If an obstacle is detected within a critical range, the buzzer is activated to alert the driver. The entire system is powered by Li-ion batteries, ensuring continuous operation.

2. Flowchart



Conclusion

The Driver-Assisted Adaptive Cruise Control (DACC) system developed in this project successfully demonstrates how real-time processing and embedded systems can be combined to improve vehicle safety and responsiveness. By using an Arduino as the sensing and actuation unit and QNX as the main decision-making system, the project achieves a clear separation of tasks, resulting in efficient and reliable operation.

The integration of Python as a communication bridge using TCP/IP client-server architecture ensures smooth and continuous data exchange between the hardware and the real-time system. The implementation shows that distance-based adaptive speed control can be effectively achieved using simple sensors and well-structured communication.

The system was able to respond appropriately under different conditions by maintaining speed at safe distances and reducing or stopping the vehicle when obstacles were detected within a critical range. The inclusion of alert mechanisms further enhances driver awareness and system safety.